

About Redis:

At its absolute core, **Redis (Remote Dictionary Server)** is simply a giant `std::unordered_map` that lives in your RAM, wrapped inside the exact TCP network server you just built.

When you build a standard web application, your main database (like PostgreSQL or MySQL) stores data on a hard drive (SSD/HDD). Reading from a hard drive is incredibly slow compared to the CPU. If a million users request the same profile picture or leaderboard score simultaneously, the database will choke and crash.

Redis sits between your backend server and your slow database.

Because Redis stores everything in RAM, it responds in microseconds. If a backend server needs data, it checks Redis first. If the data is there (a "Cache Hit"), it returns it instantly. If not (a "Cache Miss"), it fetches it from the slow database, gives it to the user, and *copies* it into Redis so the next 999,999 users get it instantly.

How Redis is Built (The Engineering Magic)

You already know the pieces required to build it. Here is how real-world Redis works under the hood:

1. The Network Layer (What you just built)

Redis uses the exact **Event Loop** (`poll` / `epoll`) you wrote for your Chat Room. It maintains thousands of open, non-blocking TCP connections from various backend servers.

2. The Concurrency Model (The Plot Twist)

Real Redis (until very recently) is **single-threaded** for command execution.

Because RAM is so fast, Redis can execute a `SET` or `GET` command in a few nanoseconds. The bottleneck is *never* the CPU; the bottleneck is the network. By using a single-threaded Event Loop, Redis completely avoids the need for `std::mutex` or `std::lock_guard`. It processes one network request at a time, linearly, making it perfectly thread-safe without the overhead of locking.

3. The Protocol (RESP)

When you connect to Redis, you don't send HTTP like a web browser. You send **RESP (Redis Serialization Protocol)**. It is a very simple, text-based binary protocol.

If you want to save a key, you send:

```
*3\r\n$3\r\nSET\r\n$4\r\nname\r\n$8\r\nAnicetus\r\n
```

Your server's job is to parse that weird string, recognize it means `SET name Anicetus`, and update the map.

4. The Data Structures

Redis is famous because it isn't just a basic Key-Value store. It supports advanced data structures that you use daily in competitive programming:

- **Strings:** standard `std::string`.
- **Lists:** Implemented internally as linked lists (like `std::list`), great for message queues.
- **Sets:** Implemented as hash tables, great for $O(1)$ lookups of unique items.
- **Sorted Sets:** Implemented using a "Skip List" (functionally similar to a `std::set` or a Red-Black tree). It keeps elements sorted by a score in $O(\log N)$ time, which is how global gaming leaderboards are built.

What You Are About to Build

For your "Mini-Redis" capstone, you will build a C++ server that:

1. Listens for TCP connections using `poll()`.
2. Reads raw text commands like `SET mykey 100` and `GET mykey`.
3. Parses the text.
4. Executes the command on an internal `std::unordered_map<std::string, std::string>`.
5. Writes the response `OK` or `100` back to the client.

Once you have the single-threaded Event Loop version working, we will actually break from the official Redis design and inject your **Thread Pool** to make a *Multi-threaded* Key-Value store, forcing you to use your `std::shared_mutex` (Read-Write locks) to protect the map.

To test your Mini-Redis server like a systems engineer, you need to verify two things: **Logic** (do `SET` and `GET` work?) and **Concurrency** (can multiple people use it at the same time without it crashing?).

Here is your exact testing protocol using macOS's built-in tools.

Step 1: Compile with Warnings

Open your terminal and compile the code. We want to ensure no hidden bugs are lurking.

```
Bash
```

```
g++ -std=c++20 -Wall -Wextra main.cpp -o miniredis
```

Step 2: Boot the Server

Run the executable.

```
Bash
```

```
./miniredis
```

(You should see [System] Mini-Redis Booted on Port 8080)

Step 3: The Multi-Client State Test

You need to prove that the server maintains a single, shared state across different connections.

1. **Open Terminal Tab 1 (Client A)** and connect:
2. Bash

nc localhost 8080

- 3.
- 4.
5. **Open Terminal Tab 2 (Client B)** and connect:
6. Bash

nc localhost 8080

- 7.
- 8.

Now, execute this exact sequence:

- **In Client A, type:**
SET my_college VIT
(Press Enter. Server should instantly reply: OK)
- **In Client B, type:**
GET my_college
(Press Enter. Server should instantly reply: VIT)
- **In Client A, type:**
GET fake_key
(Press Enter. Server should reply: (nil))
- **In Client B, type:**
DELETE my_college
(Press Enter. Server should reply: ERROR because we haven't built a DELETE command yet)

If this works, you have successfully built a functional, single-threaded, non-blocking in-memory database!

In detailed Requirements and architecture of Multithreaded Mini-Redis:

Right now, your server is completely safe because only one thread (the main thread) ever touches the `std::unordered_map`.

But what if we want to process 100,000 requests per second using all your CPU cores? We need to inject the **Thread Pool** from Phase 2.

If we do that, Thread 1 might try to write to the map (SET) while Thread 2 is trying to read from it (GET). If a hash map resizes its internal buckets while another thread is reading it, your server will **Segmentation Fault** instantly.

The CP Solution vs. The Systems Solution

In basic multithreading, you would just slap a `std::mutex` on the map.

C++

```
std::mutex mtx;

void set() {
    std::lock_guard<std::mutex> lock(mtx);
    mp[key] = val;
}

std::string get() {
    std::lock_guard<std::mutex> lock(mtx);
    return mp[key];
}
```

Why this is terrible for a database: If 10,000 users are calling GET, they aren't changing the data. They shouldn't have to wait in line for each other! A standard mutex forces everyone to wait, making your multithreaded server *slower* than your single-threaded one.

The Real Solution: `std::shared_mutex` (Read-Write Lock)

C++17 introduced the Read-Write lock.

- **Readers (GET):** Can access the map simultaneously. 10,000 threads can read at the exact same time.
- **Writers (SET):** Require *exclusive* access. If a thread wants to write, it locks out all readers and other writers until it is done.

Here is the exact implementation to upgrade your Mini-Redis map:

C++

```
#include <shared_mutex> // The C++17 library you need
```

```
class ThreadSafeCache {
```

```
private:
```

```
    std::unordered_map<std::string, std::string> mp;
```

```
    mutable std::shared_mutex rw_mtx; // Mutable so we can lock it inside const functions
```

```
public:
```

```
    // SET operation: Needs EXCLUSIVE access
```

```
    void set(const std::string& key, const std::string& val) {
```

```
        // std::unique_lock locks out EVERYONE (readers and writers)
```

```
        std::unique_lock<std::shared_mutex> exclusive_lock(rw_mtx);
```

```
        mp[key] = val;
```

```
    }
```

```
    // GET operation: Needs SHARED access
```

```
    std::string get(const std::string& key) const {
```

```
        // std::shared_lock allows OTHER shared_locks to enter simultaneously
```

```
        std::shared_lock<std::shared_mutex> shared_lock(rw_mtx);
```

```
        auto it = mp.find(key);
```

```
        if (it != mp.end()) {
```

```
            return it->second;
```

```
        }
```

```
        return "(nil)";
    }
};
```

If you replace your standard `unordered_map` with this `ThreadSafeCache` class, and pass your `handle_client` logic into your `ThreadPool`, you will have built a **Multi-threaded, High-Concurrency Key-Value Store**. This is a project that easily rivals the complexity of entry-level backend work at top companies.

Readers-Writers Problem:

Before you just type `std::shared_mutex` into your IDE, you need to understand *why* the operating system requires it.

In Computer Science, this is famously known as **The Readers-Writers Problem**. It is a foundational concept in both Operating Systems and Database Management Systems (DBMS).

Here is the deep dive into the theory, followed by an interactive simulator so you can actually see the lock in action.

The Theory: Databases and Operating Systems

Imagine a banking database. You have an account with ₹10,000.

- **Reading:** Checking your balance on your phone.
- **Writing:** Transferring ₹5,000 to a friend.

If 100 people try to *read* the balance at the exact same microsecond, there is absolutely no danger. The data isn't changing. The CPU can let all 100 threads read the memory address simultaneously.

But, if even *one* person tries to *write* (transfer money) while someone else is reading, the reader might see a corrupted, half-updated number.

How Databases Handle It (ACID Isolation)

In a DBMS like MySQL or PostgreSQL, when you run a query, the database engine automatically applies locks to the rows or tables.

- **Shared Lock (S-Lock):** When you run `SELECT * FROM users`, the DB acquires a Shared Lock. Multiple transactions can hold S-Locks on the same row at the same time.
- **Exclusive Lock (X-Lock):** When you run `UPDATE users SET balance = 0`, the DB requests an Exclusive Lock.
 - If anyone has an S-Lock, the X-Lock must *wait*.

- Once the X-Lock is acquired, it blocks *everyone* else (both new readers and new writers) until the update is finished.

How Operating Systems Handle It (File Locking)

In Unix/Linux (macOS), the OS provides a system call named `flock()` to protect files on the hard drive.

- `flock(fd, LOCK_SH)`: Acquires a Shared lock. Good for multiple processes reading a config file.
- `flock(fd, LOCK_EX)`: Acquires an Exclusive lock. Good for a process appending logs to a file.

The "Writer Starvation" Problem

There is a dangerous trap here.

Imagine a highly trafficked web server (like your Mini-Redis).

1. Thread 1 comes in to read. It gets a Shared Lock.
2. Thread 2 wants to Write. It must wait for Thread 1 to finish.
3. Thread 3 comes in to read. Since the lock is currently in "Shared" mode, Thread 3 is allowed in instantly.
4. Thread 4, 5, 6... up to 10,000 keep coming in to read.

Because there is *always* at least one active reader holding the door open, Thread 2 (the Writer) waits forever. This is called **Writer Starvation**. Modern Read-Write locks solve this by declaring: *If a Writer is waiting in the queue, no new Readers are allowed in. They must line up behind the Writer.*

Try spawning a mix of readers and writers in this simulation to see exactly how the OS manages the queues and prevents starvation.

Translating Theory to Modern C++

In C++17, the standard library finally gave us access to this OS-level feature via `<shared_mutex>`.

When you build your multithreaded Redis hash map, you use two different types of RAII lock guards to interface with the `std::shared_mutex`:

1. The Reader Guard (`std::shared_lock`)

C++

```
std::string get(const std::string& key) {
```

```

// Requests an S-Lock from the OS.

// If writers are active or waiting, this thread goes to sleep.

// If only readers are active, this thread enters instantly.

std::shared_lock<std::shared_mutex> lock(rw_mtx);

return map[key];
}

```

2. The Writer Guard (std::unique_lock)

C++

```

void set(const std::string& key, const std::string& val) {
    // Requests an X-Lock from the OS.

    // Signals to the OS: "Stop letting new readers in."

    // Waits for current readers to finish, then locks everyone out.

    std::unique_lock<std::shared_mutex> lock(rw_mtx);

    map[key] = val;
}

```

What is Lock Upgrading?

Imagine you are building the `INCR` command for your Redis server (which increments a number by 1).

The logic flows like this:

1. **Read** the current value from the map (Shared Lock).
2. Add 1 to the value.
3. **Write** the new value back to the map (Exclusive Lock).

An engineer's instinct is to "upgrade" the lock:

"I already have the Shared Lock. I'll just hold onto it, and ask the OS to upgrade it to an Exclusive Lock so I can write, then I'll release it."

The Fatal Flaw: The Upgrade Deadlock

The problem arises when **two** threads try to do this at the exact same time.

Because they are reading, the OS allows *both* Thread A and Thread B to acquire a Shared Lock simultaneously.

Now, Thread A asks to upgrade to an Exclusive Lock.

Rule of Exclusive Locks: You can only get it if **zero** other threads are reading.

Thread A goes to sleep, waiting for Thread B to drop its Shared Lock.

At the exact same millisecond, Thread B asks to upgrade to an Exclusive Lock. It goes to sleep, waiting for Thread A to drop its Shared Lock.

They are now waiting for each other forever. The server freezes permanently.

Here is a simulation of that exact failure state. Try stepping through the sequence.

The C++ Reality (`std::shared_mutex`)

Because the upgrade deadlock is so common, the C++17 `std::shared_mutex` **does not support lock upgrading**. If you try to write this code, it results in Undefined Behavior (usually an instant deadlock):

C++

```
// ANTI-PATTERN: DO NOT DO THIS
```

```
void increment(const std::string& key) {  
    std::shared_lock<std::shared_mutex> read_lock(rw_mtx); // Acquire Read  
    int val = std::stoi(mp[key]);  
  
    // Trying to grab a write lock while holding the read lock!  
    std::unique_lock<std::shared_mutex> write_lock(rw_mtx); // DEADLOCK  
    mp[key] = std::to_string(val + 1);  
}
```

(Note: The Boost C++ library has a `boost::upgrade_mutex`, but it comes with immense performance overhead and strict rules to prevent the deadlock scenario above. Standard C++ opted to leave it out entirely).

The "Drop and Reacquire" Race Condition

To avoid the deadlock in standard C++, you must drop the read lock before acquiring the write lock. But doing so introduces a **Race Condition**.

C++

```
void increment(const std::string& key) {  
    int val;  
    {  
        std::shared_lock<std::shared_mutex> read_lock(rw_mtx);  
        val = std::stoi(mp[key]);  
    } // Read lock is dropped here!  
  
    // --- RACE CONDITION WINDOW ---  
    // At this exact nanosecond, Thread B could swoop in,  
    // grab the write lock, and change the value in the map!  
    // -----  
  
    {  
        std::unique_lock<std::shared_mutex> write_lock(rw_mtx);  
        // You are about to write data based on an outdated 'val'  
        mp[key] = std::to_string(val + 1);  
    }  
}
```

If Thread A reads 10 and drops the lock, Thread B might write 99 to the map. Then Thread A grabs the write lock and blindly writes 11, destroying Thread B's update.

How Databases and C++ Systems Actually Solve This

There are two primary ways systems engineers handle operations that require both reading and writing safely.

1. Pessimistic Locking (Just grab the Write Lock immediately)

If you *know* you are going to modify the data, don't even bother with a Shared Lock. Request the Exclusive Lock from the very beginning. Yes, it briefly blocks other readers, but it is 100%

safe and prevents race conditions. This is the standard approach for a multithreaded Key-Value store.

C++

```
void increment_safe(const std::string& key) {  
    // Grab the Exclusive Lock immediately. No upgrading needed.  
    std::unique_lock<std::shared_mutex> write_lock(rw_mtx);  
    int val = std::stoi(mp[key]);  
    mp[key] = std::to_string(val + 1);  
}
```

2. Optimistic Concurrency Control (OCC)

This is what high-end distributed databases use. You read the data (without any locks), calculate the new value, and then right before you write it, you verify that nobody else changed the original data while you were calculating. If they did, you abort and retry.

In C++, this is implemented using `std::atomic::compare_exchange_weak` (CAS - Compare And Swap), which requires lock-free programming algorithms rather than standard mutexes.

For your Mini-Redis project, you should rely on **Pessimistic Locking**. When you parse an `INCR` or `SET` command, immediately acquire the `std::unique_lock`. When you parse a `GET` command, acquire the `std::shared_lock`.

The MultiThreaded Redis / The Reactor Pattern:

You are about to build the **Reactor Pattern**. This is the holy grail of high-performance backend engineering. It is the exact architecture used by Node.js, Nginx, and modern game servers.

Right now, your server is essentially doing everything itself: answering the door, taking the order, cooking the food, and handing it back. If cooking the food takes a long time, people at the door get ignored.

Already Existing Truly Multithreaded Redis / Redis Like Applications:

If your workload requires a database with a fully multithreaded core (shared-nothing architecture), consider these alternatives:

- **Dragonfly**: A drop-in Redis replacement that uses a multi-threaded architecture to achieve up to 25x higher throughput.
- **KeyDB**: A high-performance fork of Redis that uses multiple threads for the event loop and query parsing.

- **Valkey:** An open-source fork of Redis (backed by the Linux Foundation) that is actively optimizing for multi-core performance.

In the multi-threaded Reactor Pattern, we split the jobs:

1. **The Main Thread (The Host):** Runs the `poll()` loop. Its *only* job is to answer the door, take the order (read the network string), and hand the ticket to the kitchen.
2. **The Thread Pool (The Chefs):** Grabs the ticket, safely interacts with the shared database, cooks the result, writes it back to the client, and goes back to sleep.
3. **The Database (The Pantry):** Your `std::unordered_map`, protected by a `std::shared_mutex` so multiple chefs don't collide.

Here is the detailed, 3-step roadmap to upgrading your current code.

Step 1: Isolate the Database (ThreadSafeCache.hpp)

You cannot have your `mp` (hash map) floating inside your server class anymore. It needs to be its own self-contained, thread-safe entity.

- **Action:** Create a separate class for your cache.
- **The Lock:** Add a `std::shared_mutex rw_mtx;` alongside your `std::unordered_map<std::string, std::string> mp;`.
- **The Methods:** * `set(key, val)`: Must acquire a `std::unique_lock<std::shared_mutex>` (Exclusive write access).
 - `get(key)`: Must acquire a `std::shared_lock<std::shared_mutex>` (Shared read access).
- **Why:** This guarantees that 10 workers can `GET` data simultaneously without blocking, but if 1 worker calls `SET`, it safely locks everyone else out for a microsecond.

Step 2: Bring Back the ThreadPool

You need to paste the `ThreadPool` class you built earlier right above your Redis class.

- **Action:** Initialize the pool inside your `Redis::run()` method: `ThreadPool pool(4);`
- **The Hand-off:** This is where the magic happens. When your main thread's `poll()` loop detects `POLLIN` on a client socket, the main thread will `read()` the buffer. But instead of parsing the `GET` or `SET` command right there, it will bundle it into a lambda and toss it into the pool!

Step 3: The "Dispatcher" Event Loop

We have to modify your `poll()` `else` block (where you read from existing clients).

Here is the conceptual flow of the new Reactor loop:

C++

```

// 1. Main thread reads the raw network string

int bytes_read = read(sender_fd, buffer, sizeof(buffer) - 1);

// 2. If valid, capture the data and pass it to a background worker!

if (bytes_read > 0) {
    std::string raw_request(buffer);

    // The main thread instantly goes back to polling!
    // The worker thread takes over the parsing and database access.
    pool.enqueue([sender_fd, raw_request, &cache]() {

        std::string method, key, val;
        std::istream iss(raw_request);
        iss >> method;

        // ... Worker does the SET/GET using the thread-safe cache ...
        // ... Worker calls write(sender_fd, response) ...

    });
}

```

The Critical Bug You Must Avoid: The "Double Read" Trap

Because we are using `poll()`, there is a dangerous race condition.

1. Client A sends SET name Anicetus.
2. Main thread reads it, hands it to Worker 1.
3. Before Worker 1 can write the response, the Main Thread loops back around.
4. If there happens to be leftover data in the socket buffer, `poll()` will fire *again* for Client A, and the Main Thread might hand a corrupted, half-read command to Worker 2.

The Fix: Since we are building a simple protocol where one request gets one response, the easiest way to prevent this is to temporarily remove the `client_fd` from the `fds` vector right before passing it to the Thread Pool. Once the worker finishes writing the response, it adds the

`client_fd` back to a thread-safe "re-add" queue, or we just close the connection after one command (HTTP/1.0 style) to keep it simple for now.

For our Mini-Redis, let's start with the simplest approach: The worker processes the command, writes the response, and then we let the client send the next command.

Worker Pinning:

It's important to recognize the fatal flaw of a standard Thread Pool: **Non-determinism**. If you throw 3 commands into a shared pool, the OS scheduler decides which thread finishes first, destroying the strict ordering the client expects.

To solve this, we don't use complex locks or blocking sleep commands. We use an elegant routing architecture called **Worker Pinning** (also known as Sharded Queues or the Actor Model). This is exactly how modern high-performance databases like KeyDB (a multi-threaded fork of Redis) and ScyllaDB handle sequential consistency.

Worker Pinning

Instead of having **1 Shared Queue** that 4 threads fight over, we give each worker thread its own **Dedicated Queue**.

When a client connects (let's say they are given `client_fd = 7`), we hash their FD using a simple modulus operator to assign them to a specific worker:

```
target_worker = client_fd % 4 (7 % 4 = Worker 3)
```

From that moment on, **every single command** sent by `client_fd = 7` is routed exclusively to Worker 3's queue.

Because a single thread processes its queue strictly linearly (FIFO - First In, First Out), it is physically impossible for command 2 to execute before command 1.

Step 1: The `ShardedThreadPool`

We need to upgrade your `ThreadPool` class. Notice how we use an array of queues, mutexes, and condition variables—one for each thread.

Create a new file called `ShardedThreadPool.hpp`:

```
C++
```

```
#pragma once
```

```
#include <vector>

#include <thread>

#include <queue>

#include <mutex>

#include <condition_variable>

#include <functional>
```

```
class ShardedThreadPool {
private:
    struct WorkerState {
        std::queue<std::function<void()>> tasks;
        std::mutex mtx;
        std::condition_variable cv;
    };

    std::vector<std::thread> workers;
    std::vector<WorkerState> states; // One state per thread
    bool stop = false;
    size_t num_threads;

public:
    ShardedThreadPool(size_t threads) : num_threads(threads), states(threads) {
        for(size_t i = 0; i < num_threads; ++i) {
            workers.emplace_back([this, i] {
                while(true) {
                    std::function<void()> task;
                    {
                        // Worker 'i' ONLY locks its own specific mutex
                        std::unique_lock<std::mutex> lock(this->states[i].mtx);
                        this->states[i].cv.wait(lock, [this, i] {
```

```

        return this->stop || !this->states[i].tasks.empty();
    });

    if(this->stop && this->states[i].tasks.empty()) return;

    task = std::move(this->states[i].tasks.front());
    this->states[i].tasks.pop();
}
task(); // Execute sequentially
}
});
}
}

```

// New signature: Takes the client_fd so we know which queue to put it in

```

void enqueue(int client_fd, std::function<void()> task) {
    // Hash the FD to find the target worker (O(1) routing)
    size_t target = client_fd % num_threads;

    {
        std::unique_lock<std::mutex> lock(states[target].mtx);
        states[target].tasks.push(std::move(task));
    }
    states[target].cv.notify_one();
}

```

```

~ShardedThreadPool() {
    for(size_t i = 0; i < num_threads; ++i) {
        std::unique_lock<std::mutex> lock(states[i].mtx);
        stop = true;
    }
}

```



```

    }

    for(size_t i = 0; i < num_threads; ++i) {
        states[i].cv.notify_all();
    }

    for(std::thread &worker: workers) {
        worker.join();
    }
}

};

```

Step 2: Wiring it into your Reactor (main.cpp)

Now, replace your old `ThreadPool` with the new `ShardedThreadPool`. The integration in your `poll()` loop is just a one-line change.

C++

```
// ... inside Redis::run() ...
```

```
// 1. Boot the Sharded Pool
```

```
int n_threads = std::thread::hardware_concurrency();
```

```
ShardedThreadPool pool(n_threads);
```

```
while(true) {
```

```
    poll(fds.data(), fds.size(), -1);
```

```
    bool cleanup_needed = false;
```

```
    for(size_t i = 0; i < fds.size(); i++) {
```

```
        if(fds[i].revents & POLLIN) {
```

```
            // ... accept logic ...
```

```
        } else {
```

```

int sender_fd = fds[i].fd;

char buffer[4096] = {0};

int bytes_read = read(sender_fd, buffer, sizeof(buffer)-1);

if(bytes_read <= 0) {
    // ... disconnect logic ...
} else {
    std::string request(buffer);

    // 2. THE MAGIC FIX: Pass the sender_fd to the enqueue method!
    // This guarantees that all requests from sender_fd go to the same worker.
    pool.enqueue(sender_fd, [request, sender_fd, this]() {

        std::string method, key, val;
        std::istringstream iss(request);
        iss >> method;

        if(method == SET_REQUEST) {
            iss >> key >> val;

            std::string response = this->set(key, val) + "\n";
            write(sender_fd, response.c_str(), response.length());
        }
        // ... rest of your GET/DELETE logic ...
    });
}
}
}
}
// ... cleanup ...
}

```

Why this is an Engineering Masterpiece

1. **Zero Out-of-Order Execution:** Worker 2 will process the 1st command, finish it, write to the socket, and *only then* pull the 2nd command from the queue. Perfect sequential consistency for the client.
2. **Eliminated Lock Contention:** In your old pool, 4 threads fought over 1 `queue_mutex`. Now, 4 threads have their own mutexes. They never block each other when pulling tasks.
3. **CPU Cache Locality:** Because the same thread always handles the same client, the memory for that client's socket and state stays hot in that specific CPU core's L1/L2 cache.

The Final Boss (A Warning)

You have solved concurrency. But by relying on raw TCP sockets, there is one final, brutal truth of networking you will eventually face: **TCP Fragmentation**.

Right now, you assume `read(sender_fd, buffer...)` reads exactly one complete command like SET A 10.

But TCP is a stream, not a message protocol. If the client sends data too fast, one `read()` might return:

```
SET A 10\r\nSET B (half of the second command!)
```

To solve that, you would need to implement a **Session Buffer** that accumulates characters and splits them by `\n` before sending them to the pool. But for a project demonstrating multi-threading and I/O multiplexing, your current codebase is already functioning at a level that easily passes senior backend interviews.

Drop the `ShardedThreadPool` in, test it.

